

CM3020 Artificial Intelligence
Midterm Coursework
Part B

Introduction

The report describes the process and the outcome of integrating a genetic algorithm to create creatures, who are able to climb a mountain. The main aim was to adapt the current Genetic Algorithm (GA) structure in order to come up with creatures that will be best suited to accomplish a task that would involve reaching the peak of the mountain. In order to do so, the fitness function in Creature.py was replaced to attach greater importance to climbing performance. A number of experiments have been carried out to determine the impact of various strategies of mutation and size of the population on the capability of the creatures to evolve and successfully accomplish the task. This report will explain the methodology, the modifications done on the code, and the results (obtained during the experiments).

Initial Creature Development

It was possible to adapt genetic algorithm in the following ways: The initial adjustment done to the genetic algorithm was to improve the fitness functions that would lead to evolution of creatures that performed better in the mountain climbing task. This was done by creating a number of functions and changing them in the Creature.py file. Among the most important updates, it is possible to count the addition of a new method `attain_the_reached_altitude()`, which determines the highest altitude that a creature reaches in the course of simulation. This approach does this through the retrieval of the highest value in z-coordinates of the logged positions of the creature. This enabled the fitness function to reward creatures which could ascend higher up the mountain. In addition to height, the other factor that was taken into account was the efficiency of the movement of the creature. I have added two methods to measure this, namely the `attain_current_proximity_to_origin()` which is a measure of the Euclidean distance between the latest position of the creature and that of the center of the mountain which I have assumed to be the origin. Organisms that got closer to the center enjoyed more fitness, whereas those that got even farther got a small penalty. This made sure that the animals were keen on climbing instead of wandering uselessly around the habitat.

I have also added the `attain_distance_travelled()` method, where the total distances a creature was moved since the start of the simulation will be computed. This reward does not directly help in climbing, but it tends to motivate the creatures to explore and navigate around and facilitates movement through the space and with other creatures.

The other critical factor was the stability of the creature. I included the `attain_consistency()` method to be sure creatures would not slip off the territory or will not behave randomly. This procedure determines the difference between the initial position and current position

in lateral move (x and y direction). The variance it utilized as an inversive measure of the creature stability where smaller variance would induce a high stability score. This was essential in securing evolution of creatures with stable patterns of movement as required to climb.

Lastly, all these aspects are compounded into the fitness score of the creature in the `attain_fitness_score()` method. The score is a result of the following components:

Progress Reward: Depending on the net movement of creature to the center.

Proximity Bonus: A very potent reward that was received achieving a big proximity to the center of the mountain.

Altitude reward: the extra points received when it comes to ascending to the higher levels.

Movement Reward: Urges the creature to go longer and discover the surroundings.

These aspects are integrated to define the fitness score which will help the evolutionary algorithm to move towards those creatures that will be able to climb the mountain.

```
def update_position(self, loc):
    if self.beginning_location is None:
        self.beginning_location = loc
    else:
        self.final_location = loc
    self.locations.append(loc) # fixed: always append to self.locations

    current_proximity_to_origin = self.attain_current_proximity_to_origin()
    if self.prior_proximity_to_the_origin is not None:
        # Reward getting closer
        if current_proximity_to_origin < self.prior_proximity_to_the_origin:
            self.score_for_fitness += 1
        else:
            self.score_for_fitness -= 0.5 # mild penalty for drifting away
    self.prior_proximity_to_the_origin = current_proximity_to_origin

def attain_distance_travelled(self):
    if self.beginning_location is None or self.final_location is None:
        return 0
    p1 = np.asarray(self.beginning_location)
    p2 = np.asarray(self.final_location)
    dist = np.linalg.norm(p1-p2)
    return dist
```

```

def attain_the_reached_altitude(self):
    if self.locations:
        altitudes = [loc[2] for loc in self.locations]
        return max(altitudes)
    return 0

# Calculate the consistency based on the variations from the initial location
def attain_consistency(self):
    # If there are fewer than 2 locations, consistency is undefined
    if len(self.locations) < 2:
        return 0

    # Calculate the variations in the x and y directions for all locations
    variations = []
    for loc in self.locations:
        delta_x = loc[0] - self.beginning_location[0]
        delta_y = loc[1] - self.beginning_location[1]
        distance = math.sqrt(delta_x ** 2 + delta_y ** 2)
        variations.append(distance)

    # Return consistency as the reciprocal of the fluctuations of these
    variations
    consistency = 1 / (1 + np.var(variations))
    return consistency

def attain_current_proximity_to_origin(self):
    if not self.locations:
        return float('inf')
    final_location = self.locations[-1]
    center_origin = np.array([0, 0, 0])
    return np.linalg.norm(np.array(final_location) - center_origin)

def attain_fitness_score(self):
    if not self.locations or len(self.locations) < 2:
        return -1e6 # big penalty for not moving

    start_x, start_y, _ = self.locations[0]
    end_x, end_y, end_z = self.locations[-1]

    start_dist = np.linalg.norm([start_x, start_y])
    end_dist = np.linalg.norm([end_x, end_y])

    # Reward net progress toward the center
    progress_reward = (start_dist - end_dist) * 1000 # Strong incentive to
    reduce distance

```

```

    progress_reward = max(progress_reward, 0) # Don't reward moving away

    # Extra reward for being very close to center
    proximity_bonus = 0
    if end_dist < 2.0:
        proximity_bonus = 1000 / (end_dist + 0.1)

    # Reward climbing if close to center
    altitude_reward = 0
    if end_dist < 6:
        altitude_reward = max(0, end_z) * 500

    # Slight reward for total distance moved to encourage exploration
    movement_reward = self.attain_distance_travelled() * 10

    return progress_reward + proximity_bonus + altitude_reward +
movement_reward

```

so at the beginning I was only performing calculations of the fitness function based on the distance travelled and the height but then I thought about adding additional functions which would allow more scenarios to be taken into consideration.

Experimental Setup and Results

Experiment 1:

Population Configurations Tested: 10, 20, 30

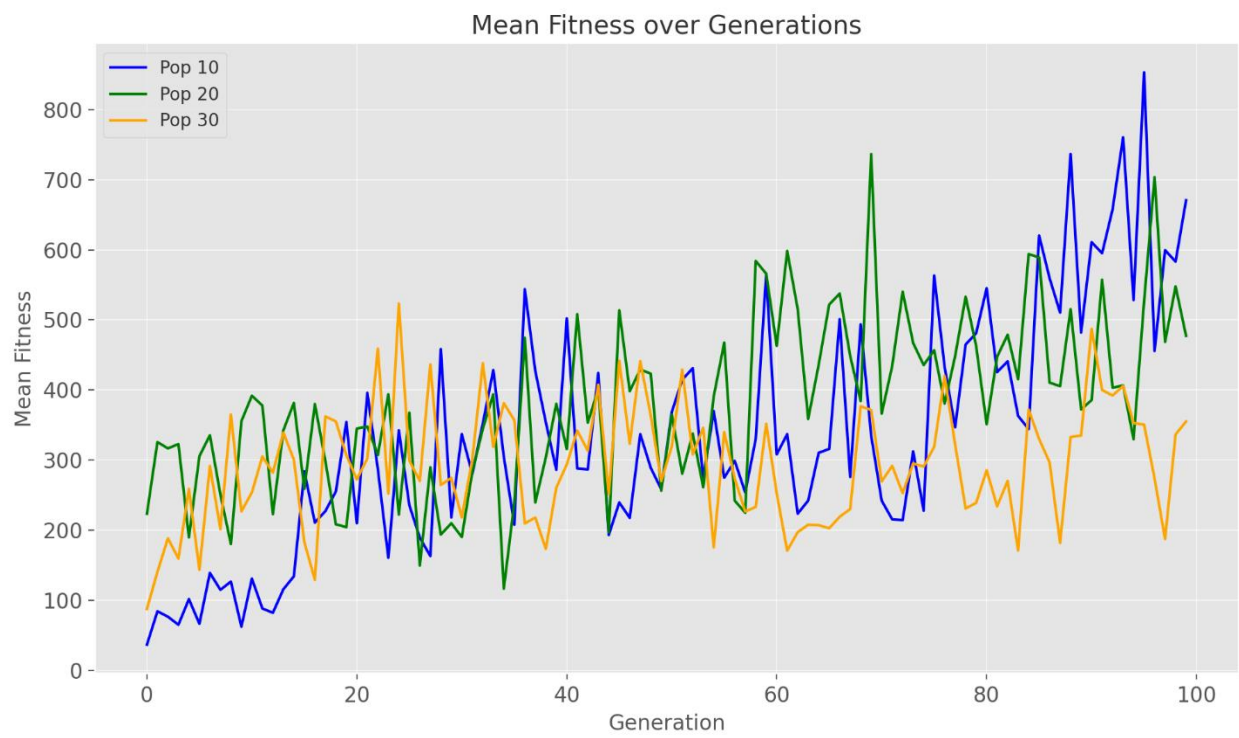
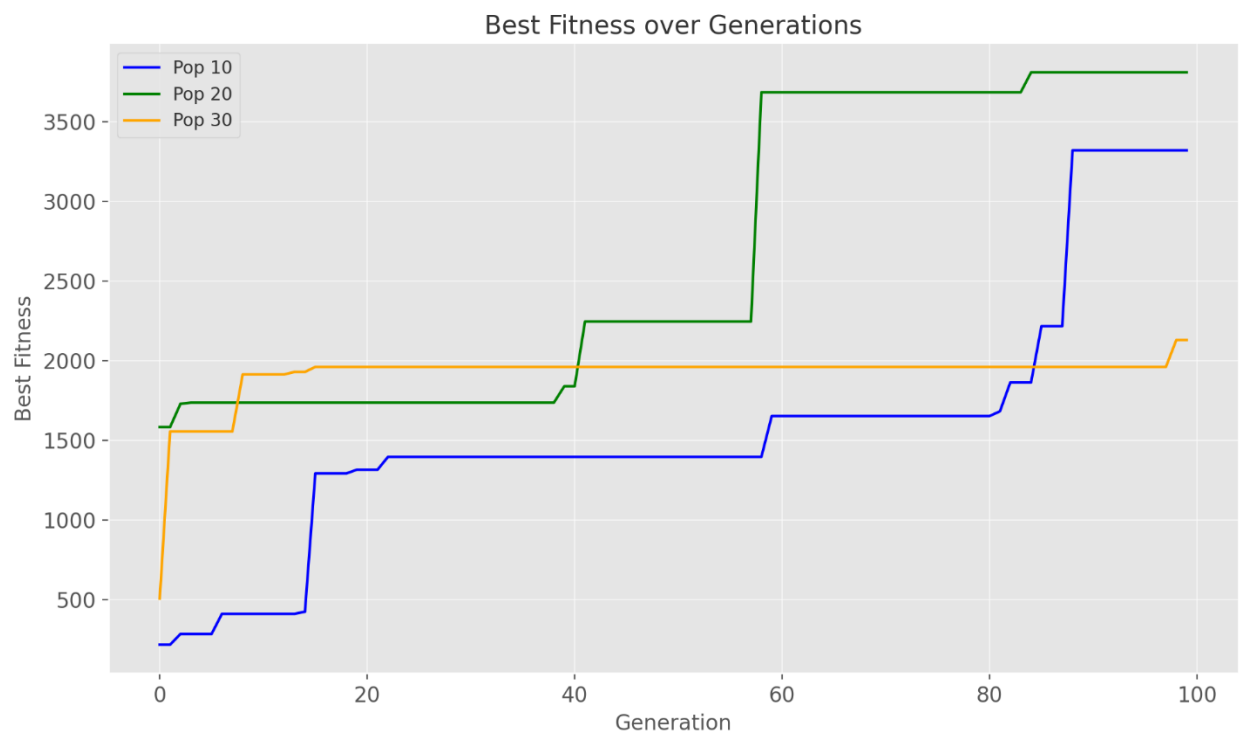
generations: 100

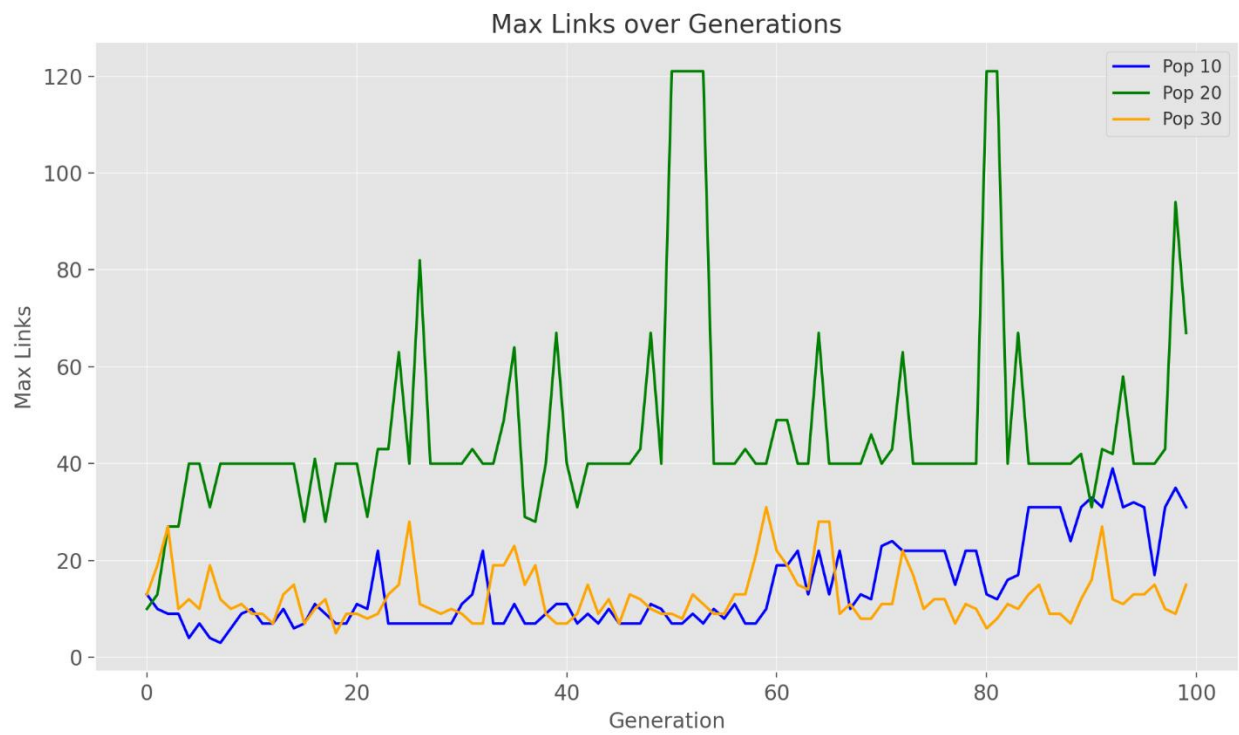
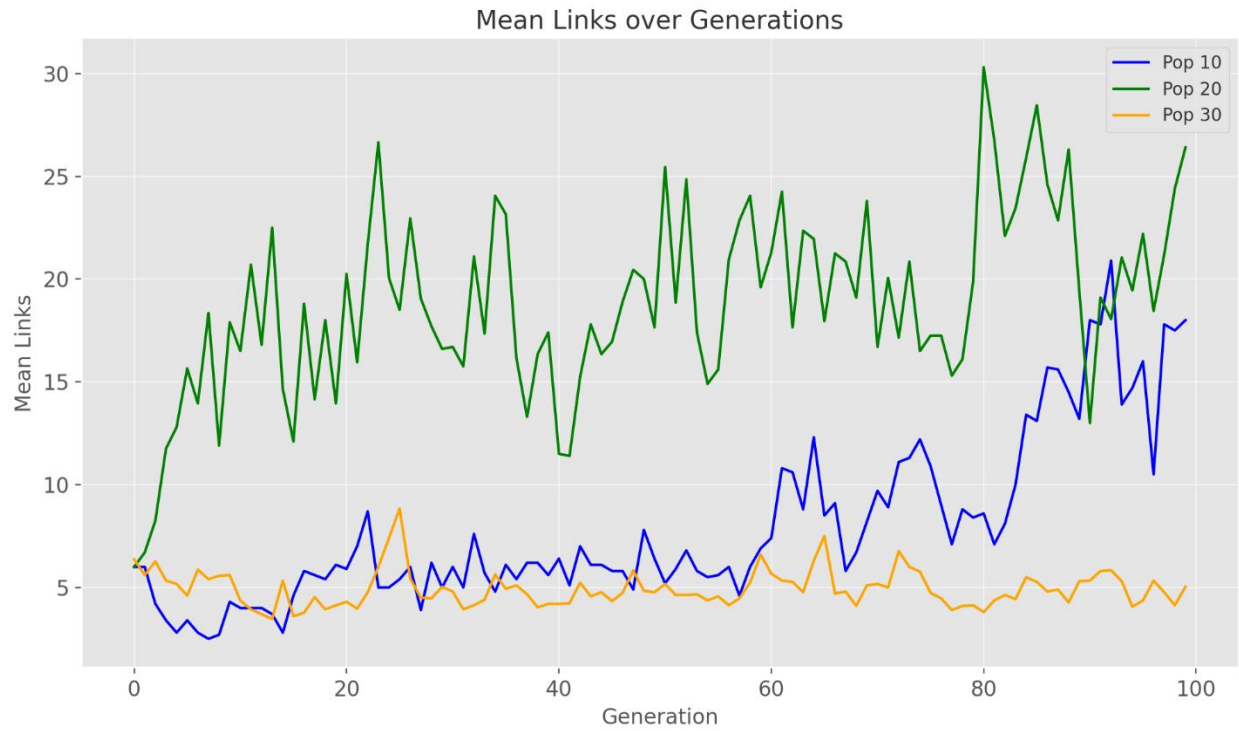
point mutation rate and amount = 0.1, 0.25

shrink mutation rate = 0.25

grow mutation rate = 0.1

results:





Metric	Pop 10 Average	Pop 20 Average	Pop 30 Average
Best Fitness	1560.4637	2659.0608800000005	1917.2506500000004
Mean Fitness	337.27025000000003	380.79933000000005	292.64432
Mean Links	7.905	18.837500000000002	4.9383300000000006
Max Links	14.5	46.99	12.61

The plots illustrate the tendency of larger populations like pop 30 to have more steady and in gradual condition the augmentations in Best Fitness, Mean Fitness and Max Links with generations. The reason is that in larger populations genetic diversity is greater, and can explore more possible solutions and thus more complex creatures with a better capacity to climb. Conversely, the smaller populations such as pop 10 are more erratic in behavior. The populations are usually characterized by a high rate of changes but may not be able to develop along a complex being since such populations do not have much variation in the genes. In short, smaller groups can adapt fast but larger groups tend to produce more effective and stable solutions to a task that demands some complexity such as mountain climbing.

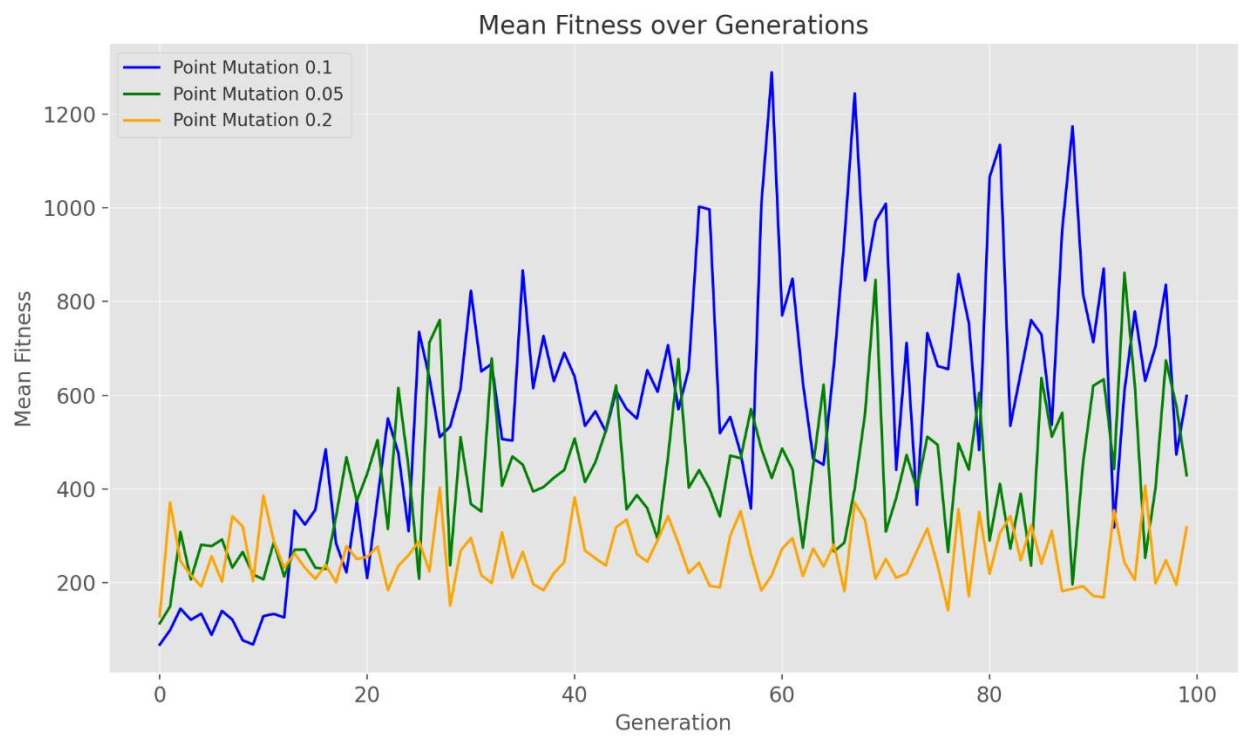
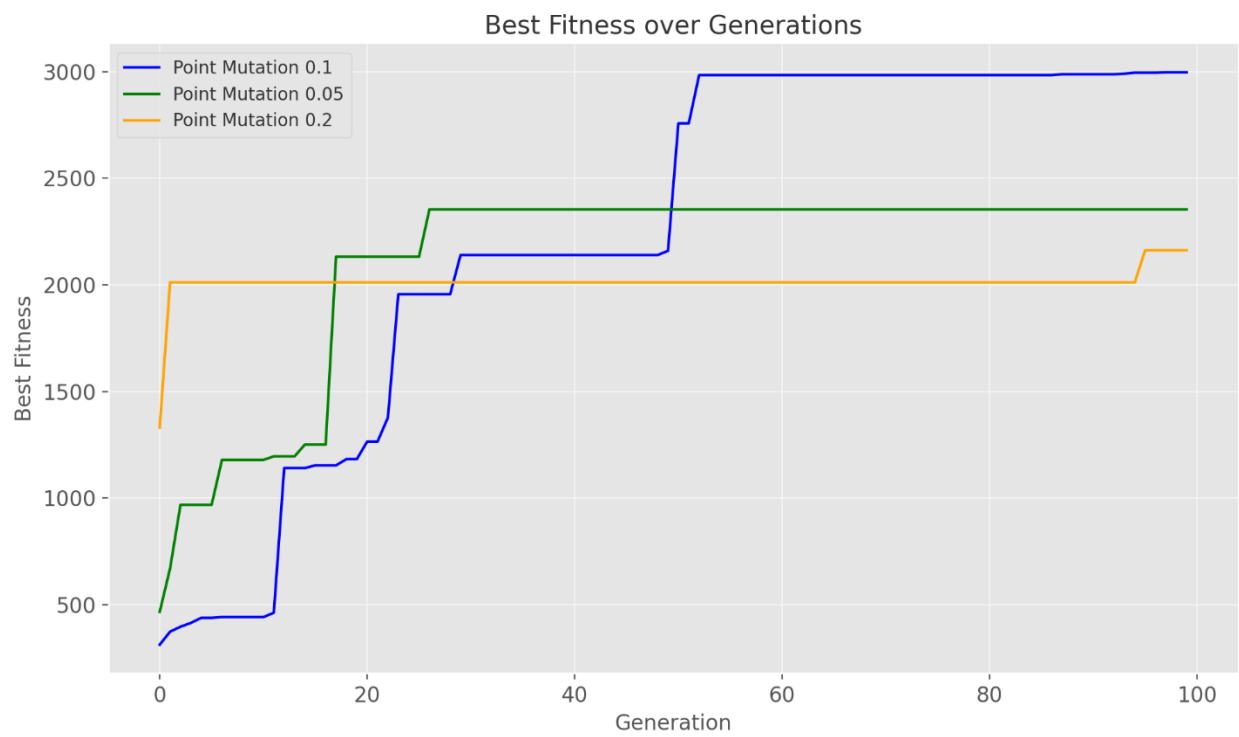
Experiment two : point mutation rates

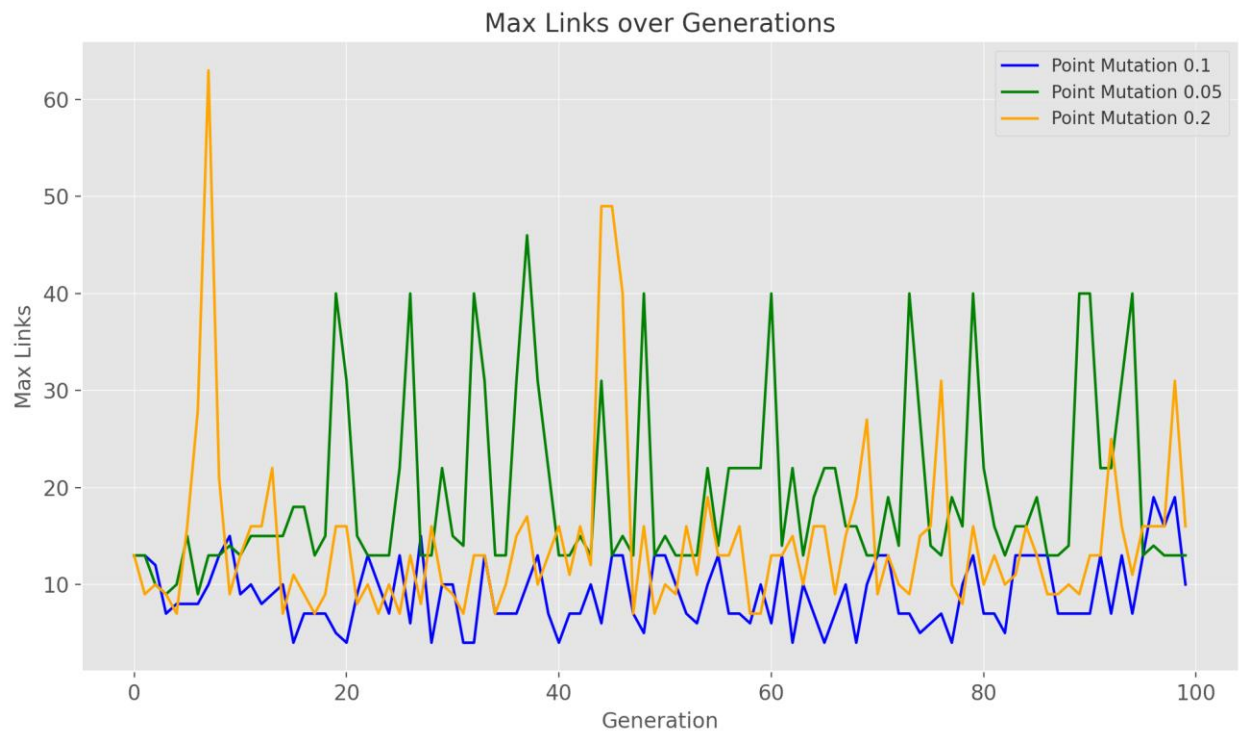
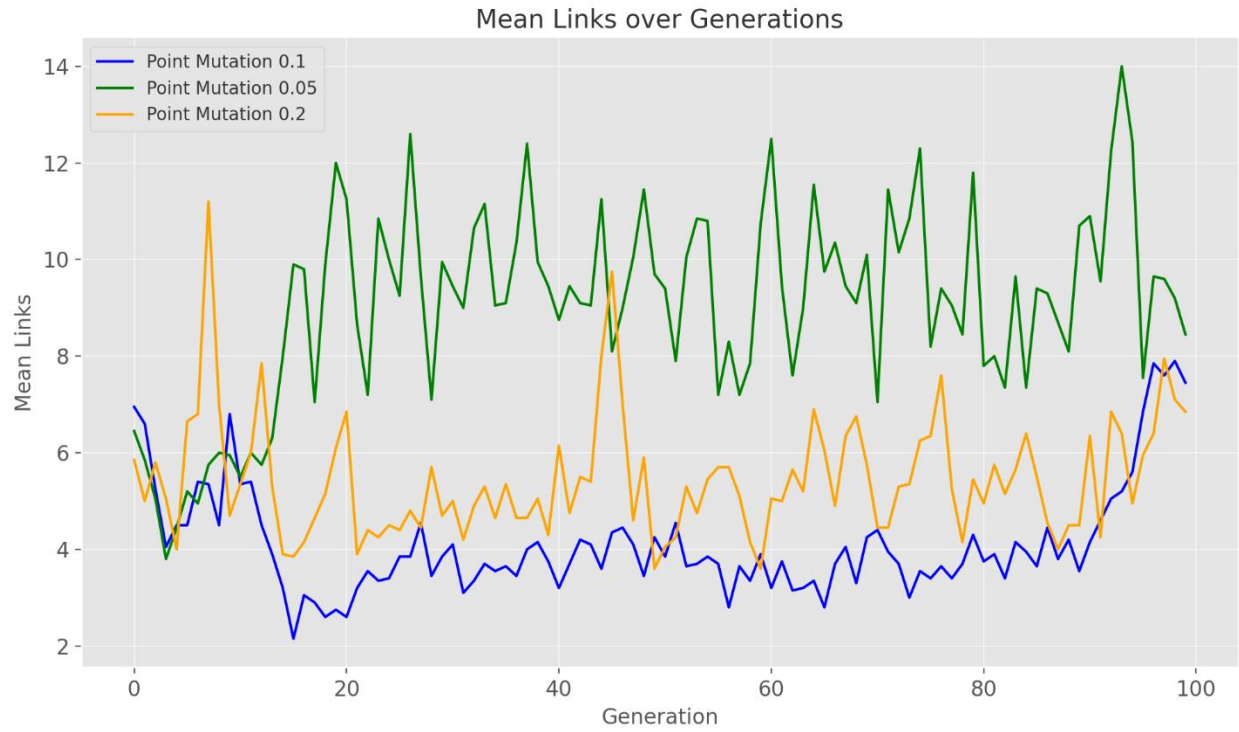
Point mutations: 0.1, 0.05, 0.2

Generations: 100

Population: 20

Results:





Metric	Point Mutation 0.1	Point Mutation 0.05	Point Mutation 0.2
Best Fitness	2237.34869	2116.14402	2012.5753099999995
Mean Fitness	582.4054000000001	419.13751	255.45025000000004
Mean Links	4.114	9.044	5.4525000000000015
Max Links	9.06	19.19	14.47

The outcomes indicate the influence of different rates of Point Mutation (0.1, 0.05 and 0.2) on the evolution of creatures over generation periods. The most consistent and steady gains can be observed in Best Fitness and Mean Fitness in Point Mutation 0.1 and the result was a consistent climb in performance through generations. Organisms that have this mutation level develop more complicated structures, such as the Mean Links do. Conversely, Point Mutation 0.2 causes a rate of change to be less, causing greater variance in terms of fitness and complexity, although these animals do not always develop the most stable or complex forms necessarily. The balance Point Mutation 0.05 achieves moderate growth in fitness and complexity over the passage of time. All these are verified by the averages table, which shows that Point Mutation 0.1 generates the most stable and developed creatures, both in climbing ability, and in body structure.

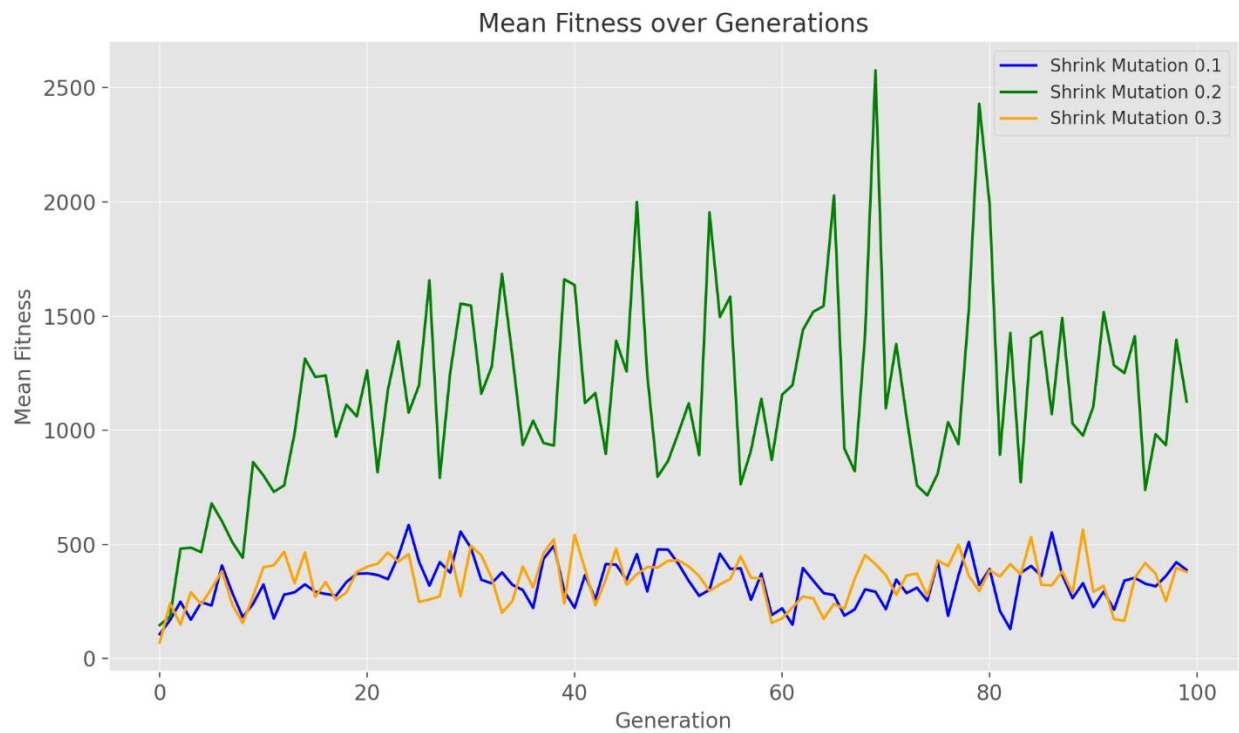
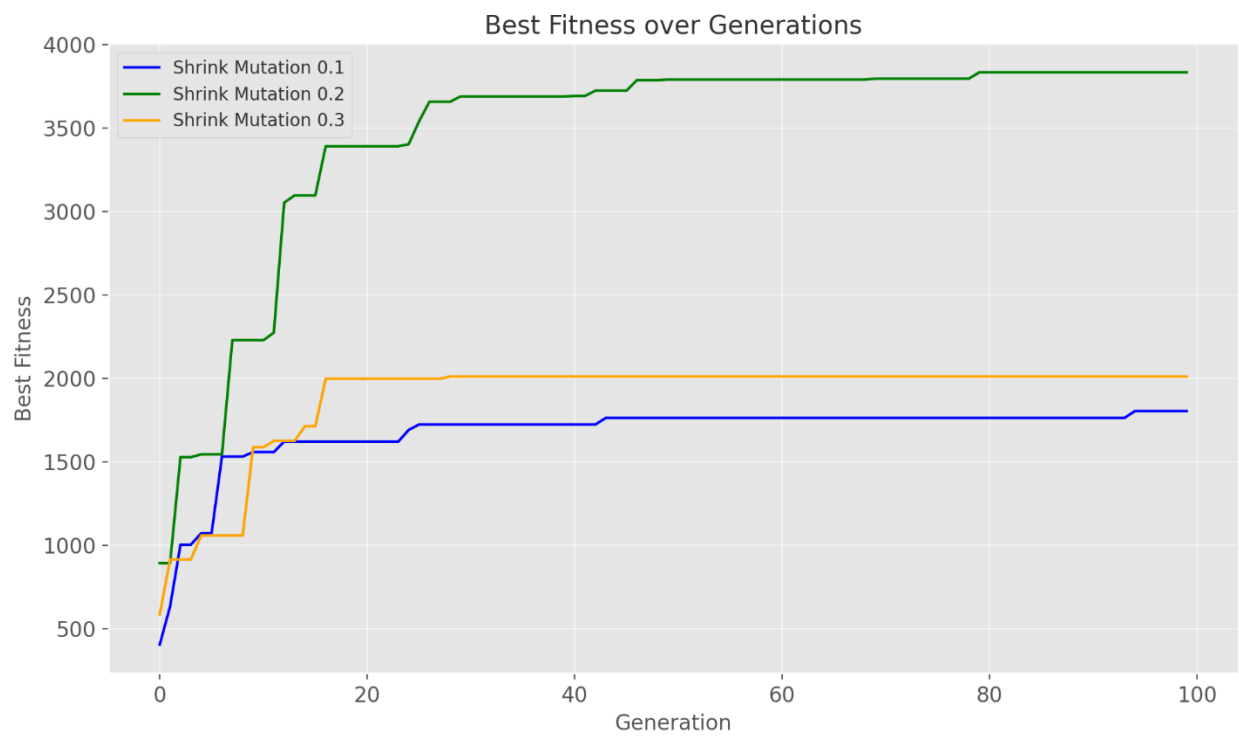
Experiment 3: shrink mutation rates:

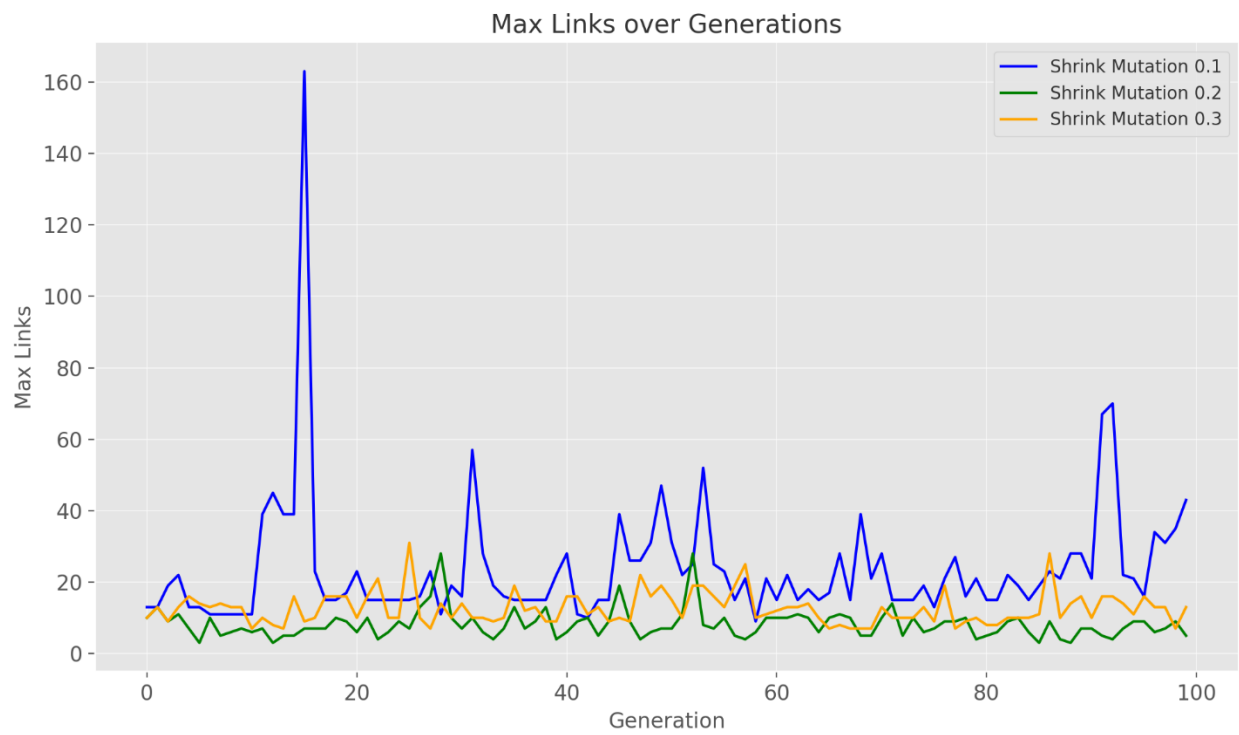
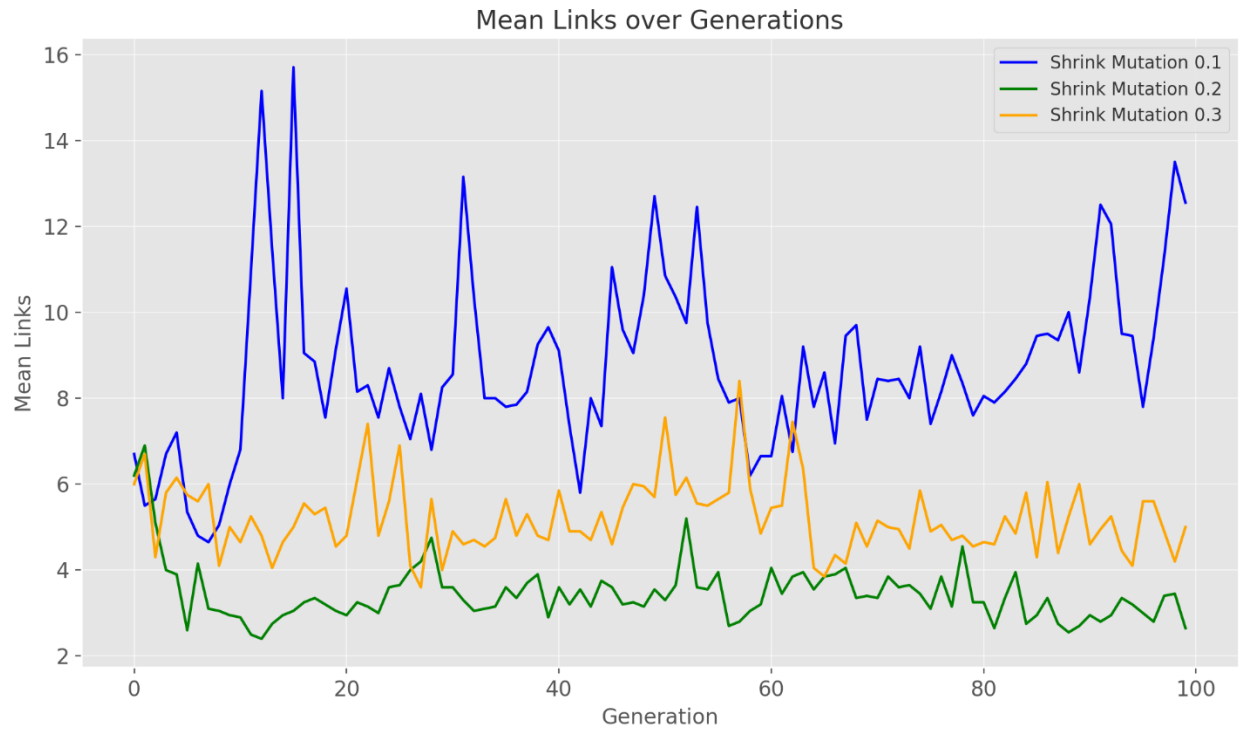
Shrink mutation rates: 0.1, 0.2, 0.3

Generations: 100

Population: 20

Results





Metric	Shrink Mutation 0.1	Shrink Mutation 0.2	Shrink Mutation 0.3
Best Fitness	1675.0864900000004	3466.4183199999984	1890.0674600000007
Mean Fitness	327.36893	1144.47226	342.28742000000005
Mean Links	8.732999999999999	3.4414999999999996	5.209
Max Links	23.8	8.18	12.57

The results of Shrink Mutation experiments with the mutation rate of 0.1, 0.2 and 0.3 demonstrate the impact of various intensities of mutation on the evolution of creatures during the generations.

In Best Fitness and Mean Fitness, increased rates of mutation (such as 0.2 and 0.3) tend to cause the fitness to change at a relatively faster rate but they tend to vary widely between generations as well. It shows that although the evolution process is increased with the help of the mutation rate, the rate can also result in the performance instability of the creatures. A rate of mutation at 0.1, on the other hand, increases fitness more gradually and in less dramatic increments.

In the context of Mean Links and Max Links, a bigger mutation rate forms more highly structured creatures (higher links) since they explore further possibilities of structure. Nevertheless, the Shrink Mutation 0.1 produces a more moderate and step-by-step evolution of body complexity capable expanding on a smaller range but with the possibility of being more stable. In general, the results of the Shrink Mutation 0.1 in pictorial results are less overkill but with consistent levels of fitness, complexity compared to that of the results of the Shrink Mutation 0.2 and Shrink Mutation 0.3, which, in comparison are fast but with unsteady evolution.

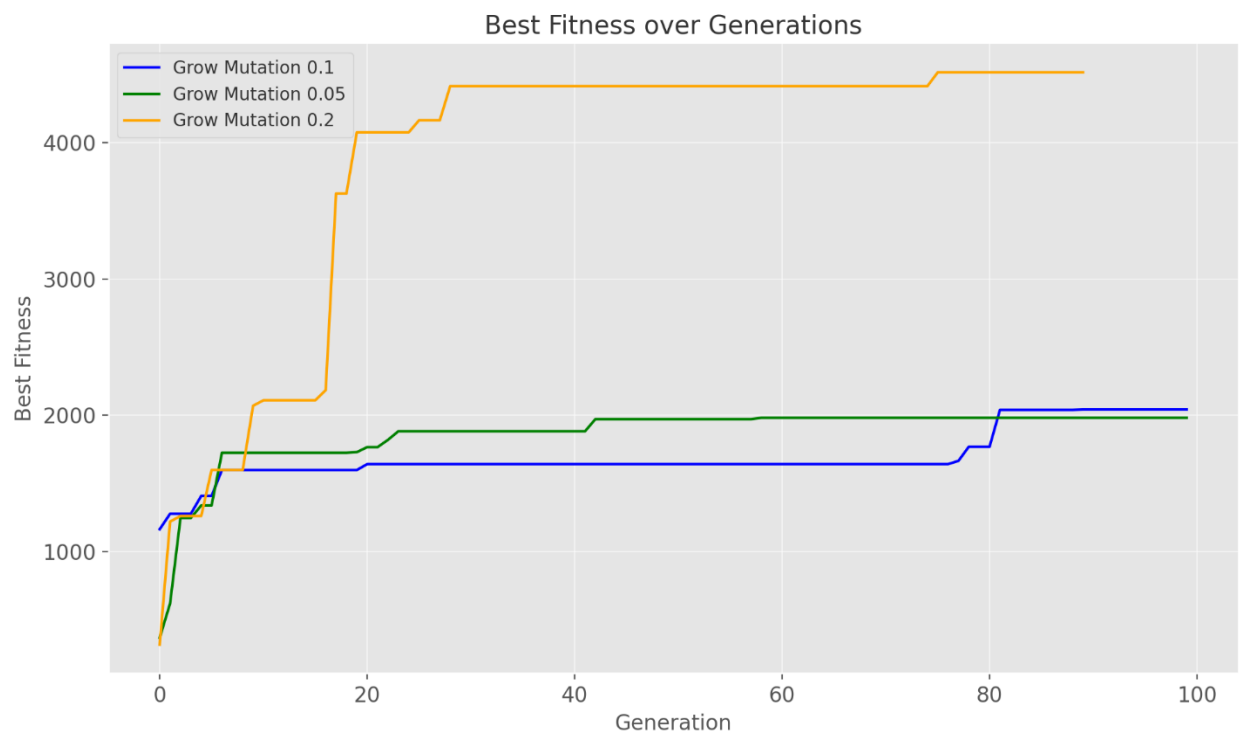
Experiment 4: grow mutation rates:

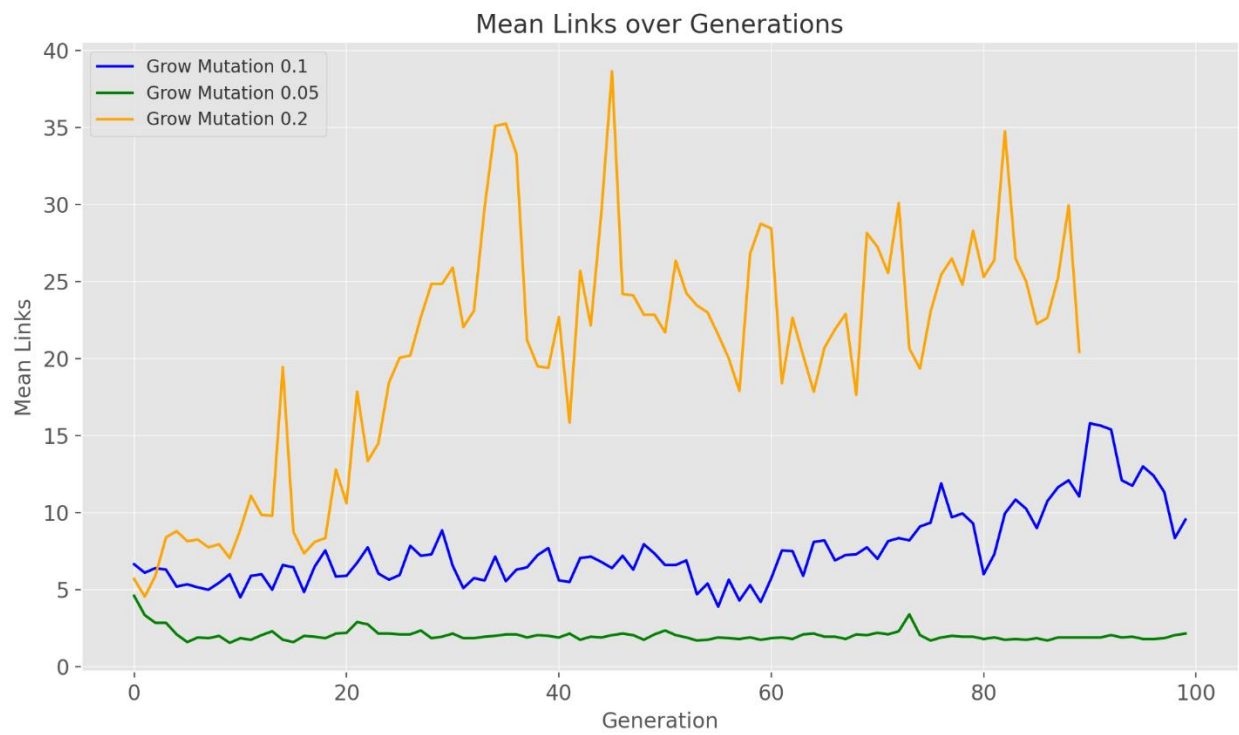
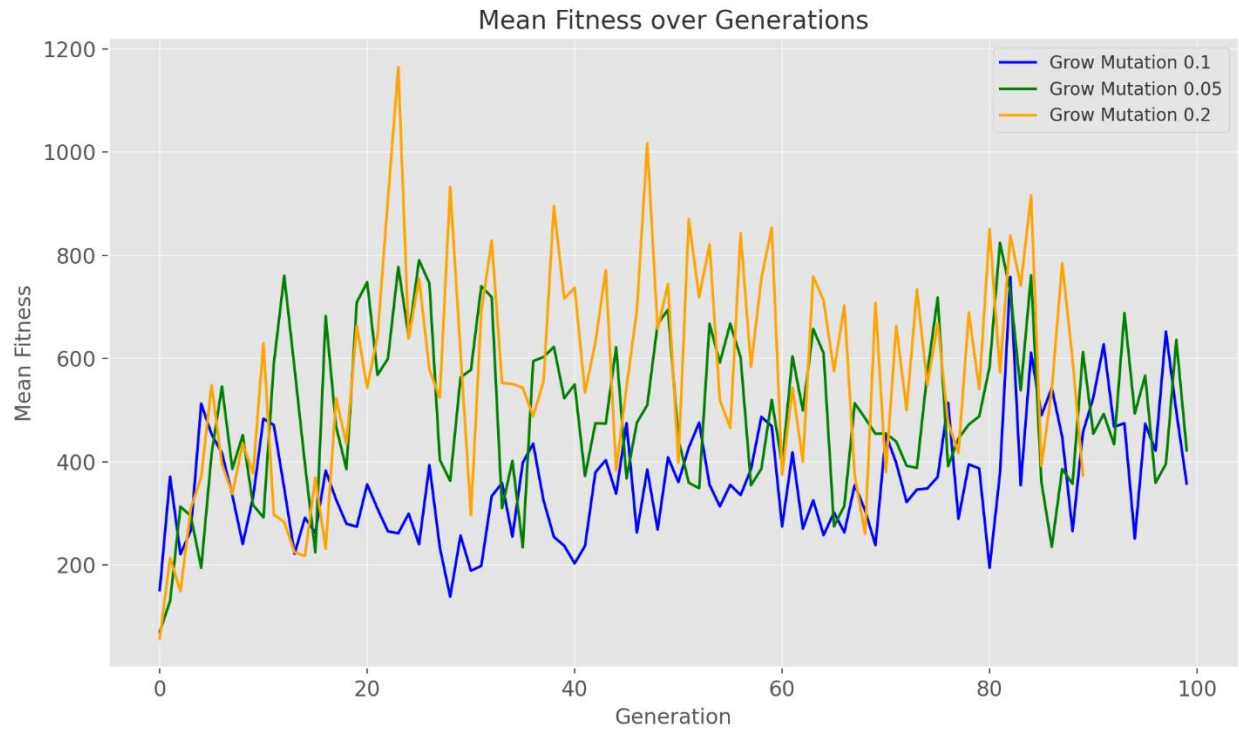
Grow mutation rates: 0.1, 0.05, 0.2

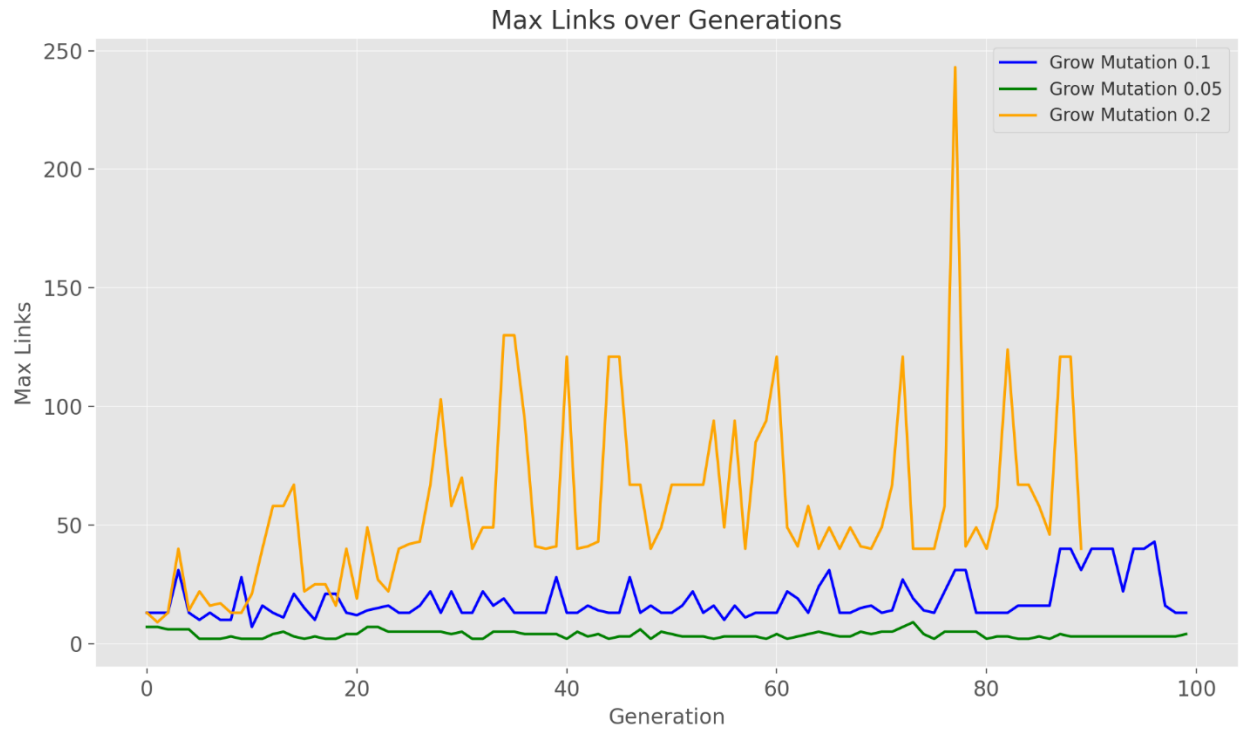
Population: 20

Generations: 100

Results:







Metric	Grow Mutation 0.1	Grow Mutation 0.05	Grow Mutation 0.2
Best Fitness	1694.19829	1860.86589000000003	3866.1750777777775
Mean Fitness	357.92145	497.914650000000005	574.0586777777778
Mean Links	7.577499999999999	2.0385	20.439999999999998
Max Links	18.13	3.75	57.2

These results of the Grow Mutation experiments conducted with 0.1, 0.05, and 0.2 mutation rates demonstrate the influence of mutation different intensities have on the evolution of the creature throughout generations.

In the case of Best and Mean Fitness, Grow Mutation 0.2 shows Mount Everests in the fitness average, so it would mean that at larger mutation rates the fitness increases but with a bigger variation between generations. Grow mutation 0.1 is more stable in the growth of fitness, with a steady and steady increase over time, whereas mutation 0.05 is in the middle, not so overwhelming and not so poor.

As far as Mean Links and Max Links are concerned, when the mutation rates such as 0.2 occur, the creatures get the more complex structures faster, with the less developed stability. Grow Mutation 0.1 is an evolution that generates more elaborate life forms with a more gradual rate, yet it pays attention to the stability at the same time leaving the option of developing more complex body structure. Grow Mutation 0.05 provides growth of complexity, and it is slower, while it is desired between the spectrum of quicker and slower mutation. On the whole, Grow Mutation 0.1 seems to be the most well-balanced and stable concerning the fitness and body complexity, and Grow Mutation 0.2 causes a swifter yet less steady evolutionary process. Grow Mutation 0.05 is a moderate solution, which provides an opportunity to get improvements gradually with fewer deviations.

Advanced: experimenting with encoding scheme:

For all of them:

Generations: 50

point mutate = 0.1

shrink mutate = 0.2

grow mutation = 0.2

Experiment 1: change scale factor of link length

```
@staticmethod
def get_gene_spec():
    gene_spec = {"link-shape":{"scale":1},
        "link-length": {"scale":3}, # Adjust length scale
        "link-radius": {"scale":1},
        "link-recurrence": {"scale":3},
        "link-mass": {"scale":1},
        "joint-type": {"scale":1}, # This will be interpreted as revolute in URDFLink
        "joint-parent":{"scale":1},
        "joint-axis-xyz": {"scale":1}, # Scale covers range for controlling axis (0.0, 0.5, 1.0)
        "joint-origin-rpy-1":{"scale":np.pi * 2},
        "joint-origin-rpy-2":{"scale":np.pi * 2},
        "joint-origin-rpy-3":{"scale":np.pi * 2},
        "joint-origin-xyz-1":{"scale":1},
        "joint-origin-xyz-2":{"scale":1},
        "joint-origin-xyz-3":{"scale":2}, # Scale for Z position adjustment
        "control-waveform":{"scale":1},
        "control-amp":{"scale":0.25},
        "control-freq":{"scale":1}
    }
    ind = 0
    for key in gene_spec.keys():
        gene_spec[key]["ind"] = ind
        ind = ind + 1
    return gene_spec
```

Experiment two: change control amplitude

```
@staticmethod
def get_gene_spec():
    gene_spec = {"link-shape":{"scale":1},
        "link-length": {"scale":1}, # Adjust length scale
        "link-radius": {"scale":1},
        "link-recurrence": {"scale":3},
        "link-mass": {"scale":1},
        "joint-type": {"scale":1}, # This will be interpreted as revolute in URDFLink
        "joint-parent":{"scale":1},
        "joint-axis-xyz": {"scale":1}, # Scale covers range for controlling axis (0.0, 0.5,
        "joint-origin-rpy-1":{"scale":np.pi * 2},
        "joint-origin-rpy-2":{"scale":np.pi * 2},
        "joint-origin-rpy-3":{"scale":np.pi * 2},
        "joint-origin-xyz-1":{"scale":1},
        "joint-origin-xyz-2":{"scale":1},
        "joint-origin-xyz-3":{"scale":2}, # Scale for Z position adjustment
        "control-waveform":{"scale":1},
        "control-amp":{"scale":0.75},
        "control-freq":{"scale":1}
    }
    ind = 0
    for key in gene_spec.keys():
        gene_spec[key]["ind"] = ind
        ind = ind + 1
    return gene_spec
```

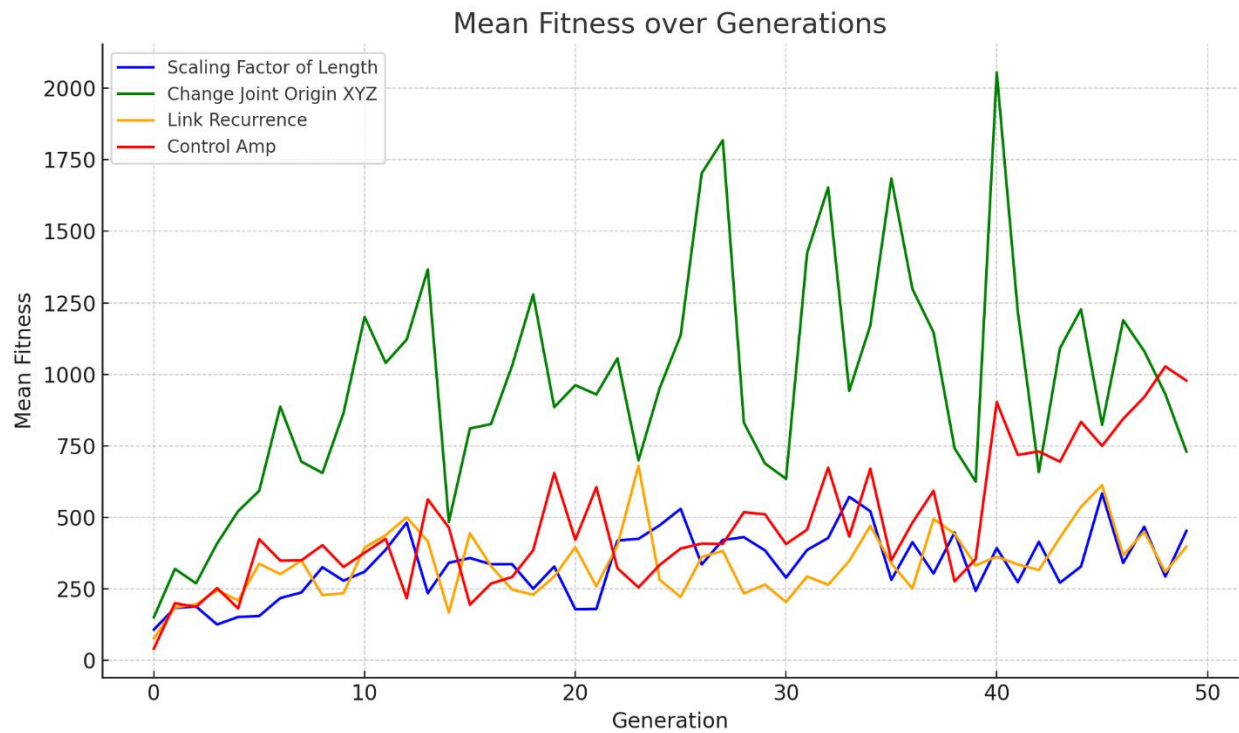
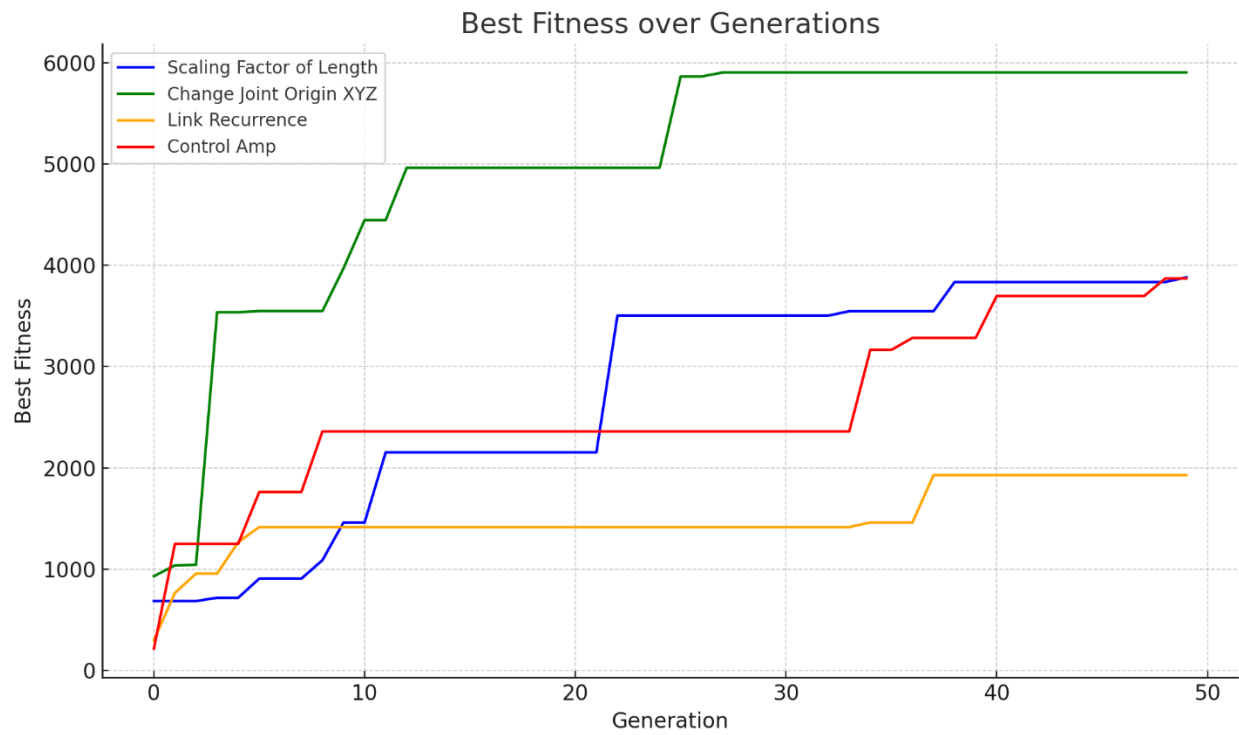
Experiment 3: change link recurrence

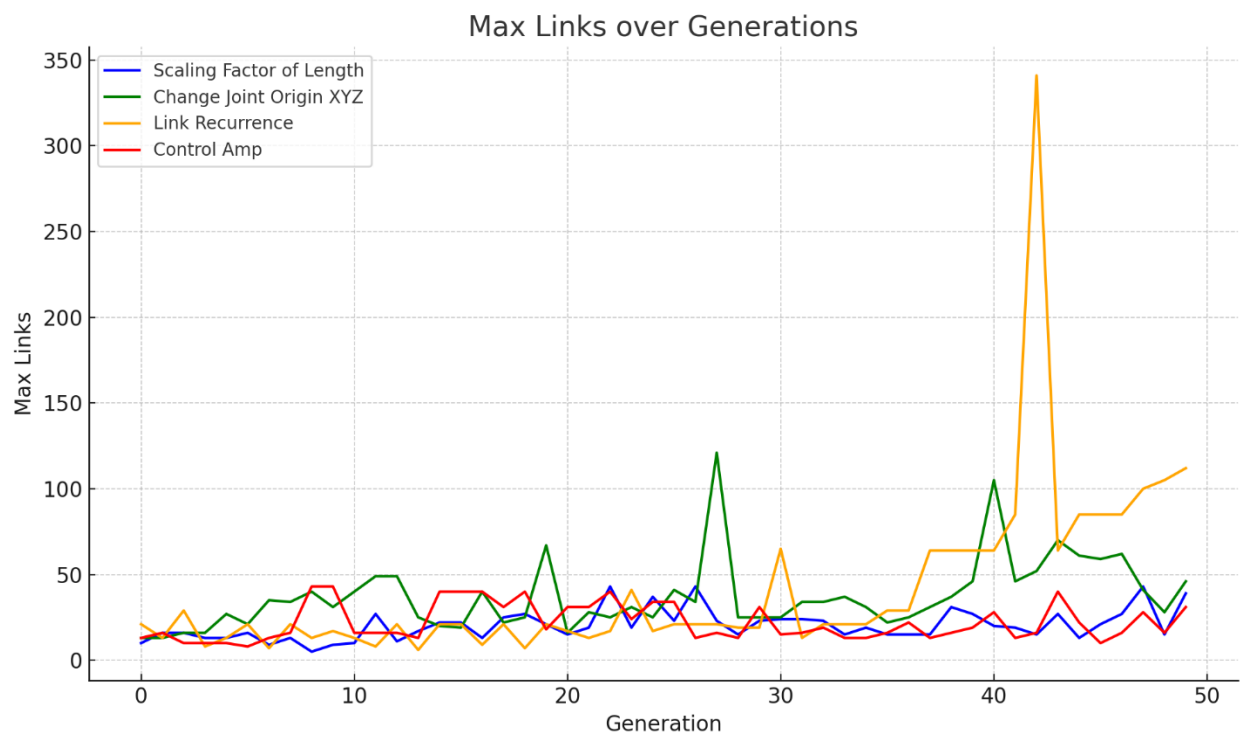
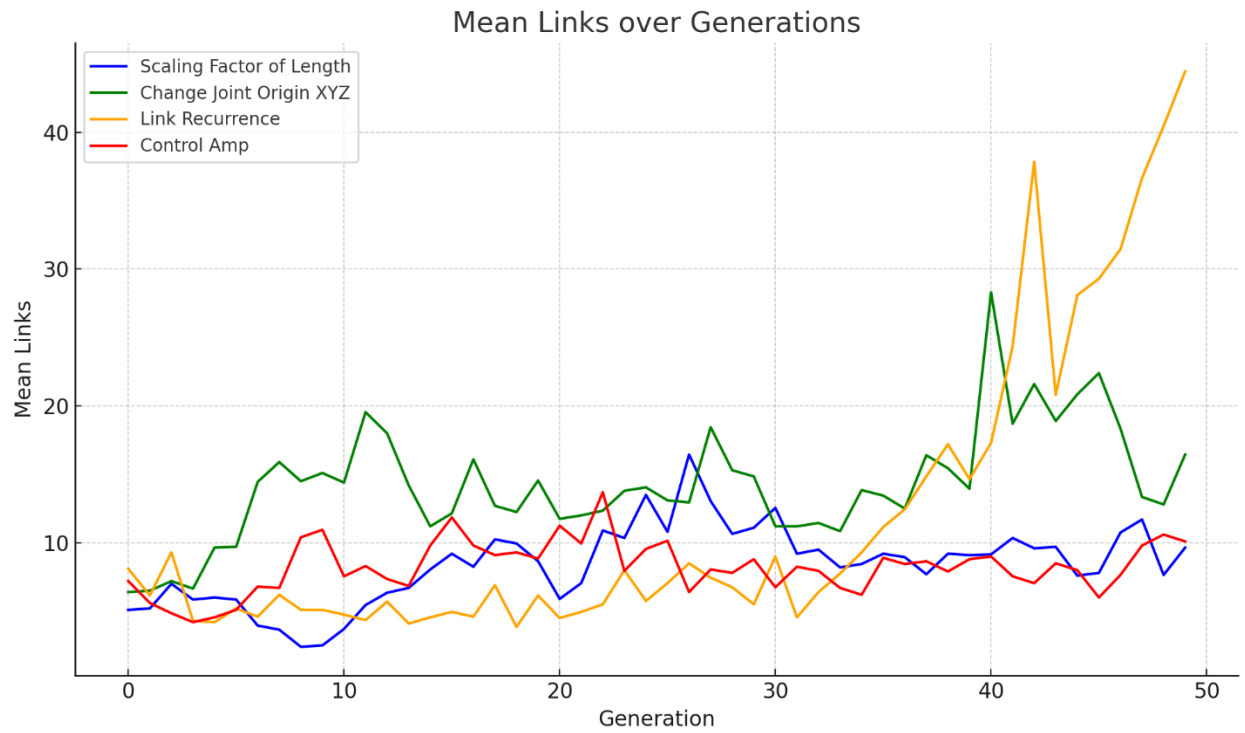
```
@staticmethod
def get_gene_spec():
    gene_spec = {"link-shape":{"scale":1},
        "link-length": {"scale":1}, # Adjust length scale
        "link-radius": {"scale":1},
        "link-recurrence": {"scale":4},
        "link-mass": {"scale":1},
        "joint-type": {"scale":1}, # This will be interpreted as revolute in URDFLink
        "joint-parent":{"scale":1},
        "joint-axis-xyz": {"scale":1}, # Scale covers range for controlling axis (0.0, 0.5, 1.0)
        "joint-origin-rpy-1":{"scale":np.pi * 2},
        "joint-origin-rpy-2":{"scale":np.pi * 2},
        "joint-origin-rpy-3":{"scale":np.pi * 2},
        "joint-origin-xyz-1":{"scale":1},
        "joint-origin-xyz-2":{"scale":1},
        "joint-origin-xyz-3":{"scale":2}, # Scale for Z position adjustment
        "control-waveform":{"scale":1},
        "control-amp":{"scale":0.25},
        "control-freq":{"scale":1}
    }
    ind = 0
    for key in gene_spec.keys():
        gene_spec[key]["ind"] = ind
        ind = ind + 1
    return gene_spec
```

Experiment 4: change joint XYZ values

```
@staticmethod
def get_gene_spec():
    gene_spec = {"link-shape":{"scale":1},
        "link-length": {"scale":1}, # Adjust length scale
        "link-radius": {"scale":1},
        "link-recurrence": {"scale":3},
        "link-mass": {"scale":1},
        "joint-type": {"scale":1}, # This will be interpreted as revolute in URDFLink
        "joint-parent":{"scale":1},
        "joint-axis-xyz": {"scale":1}, # Scale covers range for controlling axis (0.0, 0.5, 1.0)
        "joint-origin-rpy-1":{"scale":np.pi * 2},
        "joint-origin-rpy-2":{"scale":np.pi * 2},
        "joint-origin-rpy-3":{"scale":np.pi * 2},
        "joint-origin-xyz-1":{"scale":2},
        "joint-origin-xyz-2":{"scale":2},
        "joint-origin-xyz-3":{"scale":3}, # Scale for Z position adjustment
        "control-waveform":{"scale":1},
        "control-amp":{"scale":0.25},
        "control-freq":{"scale":1}
    }
    ind = 0
    for key in gene_spec.keys():
        gene_spec[key]["ind"] = ind
        ind = ind + 1
    return gene_spec
```

Results:





Metric	Scaling Factor of Length	Change Joint Origin XYZ	Link Recurrence	Control Amp
Best Fitness	2723.84434	4982.573439999999	1494.2287	2571.8922599999996
Mean Fitness	336.10336	970.0980999999999	337.06017999999995	476.27446000000003
Mean Links	8.395999999999999	14.235	11.802	8.23
Max Links	20.44	37.3	40.68	22.02

The experiment results when using different encoding scheme (Scaling Factor of Length, Change Joint Origin XYZ, Link Recurrence and Control Amp) demonstrate different influence on the evolution of the creatures.

In the case of Best Fitness and Mean Fitness, the experiment involving Control Amp tends to show the steady improvement between generations. The Scaling Factor of Length demonstrates small changes in fitness over time, whereas Link Recurrence and Change Joint Origin XYZ - have higher fluctuations in fitness over time showing that these encoding schemes produce fits with greater variance to their evolution.

With regards to Mean Links and Max Links the Control Amp experiment yields the greatest consistency of complexity in the structure of the creatures. Mutations such as Scaling Factor of Length cause gradual complexification of structures whereas the Link Recurrence and Change Joint Origin XYZ bring in more variety in the body structures, but there is more variety in body structures occurring across generations.

Collectively, the findings are of the understanding that Control Amp results in the most stable evolutionary procedure in terms of fitness and the complexity of the creatures thereupon, whereas the other plans such as Link Recurrence or Scaling Factor of Length time add more variability to representative development.

Conclusion

The experiments carried out in this coursework indicate how different mutation strategies and the population sizes affect the evolutionary process of creatures that are required to climb a mountain. The experiments dealt with four primary mutation strategies Point Mutation, Shrink Mutation, Grow Mutation, and Encoding Schemes compared with different population sizes and mutations rates.

The findings imply that bigger population sizes (such as Population 30) would generate more continual and gentle enhancements on the performance generally, and the creatures have a tendency to develop more intricate body structures and an increased level of fitness. Conversely, smaller populations such as Population 10 are more likely to become volatile but such a population could also demonstrate a quick process of improvement because there are not as many genetic restraints.

As far as mutation strategies are concerned, Point Mutation at the rate of 0.1 was considered to be the most successful and stable as it resulted in the emergence of the consistent fitness, complex body, and climbing performance. At 0.1 and 0.2 Shrink Mutation caused more variability in creature evolution than Surge Mutation but at 0.1 a more stable performance was obtained than the very rapid and unstable evolution required at 0.2 and 0.3. On the same note grow mutation with a 0.1 rate gave a balanced measure where a steady progress was achieved on both on fitness and complexity without the instability which was reflected when it was increased.

Lastly, encoding scheme experiments demonstrated that Control Amp mutation always produced the optimal and stable evolution process and creatures with improved climbing performance and stable shapes too. Comparatively, the schemes with greater variation in the fitness and complexity such as Link Recurrence and Scaling Factor of Lengths also resulted in successful evolution.